

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 8

Operating Systems

Topic: The Same Calls Inside Your Own OS

run and pingpong in xv6

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**

1.1 Processes in xv6 4

1.2 Pipes in xv6 4

2 run, Inside xv6 **5**

2.1 The Program 5

2.2 Running It 5

3 pingpong **8**

3.1 Implementation 8

4 Optional: primes **10**

Conclusions **12**

INTRODUCTION

In Laboratory Work No. 7 I ran `fork`, `exec` and `wait` on Linux. The purpose of this laboratory work is to run the same program inside xv6, the small operating system built during this semester, and to see that the system calls are the same. There is no glibc in xv6: only the system calls of the kernel I compiled and a tiny user library. In the second part two processes talk to each other through pipes (`pingpong`), and in the optional part a chain of processes connected by pipes computes prime numbers (`primes`).

I used my `xv6-riscv` tree from the previous labs. The submitted files are `user/run.c`, `user/pingpong.c`, `user/primes.c` and the `Makefile` with the new `UPROGS` lines:

```
UPROGS=\
        $U/_cat\
        ...
        $U/_zombie\
        ...
        $U/_sync\
        $U/_run\
        $U/_pingpong\
        $U/_primes\
```

Listing 1: The new lines in UPROGS (each starts with a TAB, nothing after the backslash)

1 SHORT THEORY

1.1 Processes in xv6

xv6 offers the same process model as Unix. `fork()` copies the page table, the registers and the table of open files of the calling process into a new `struct proc` and returns 0 in the child and the child's PID in the parent. `exec(path, argv)` reads an ELF file from the file system, builds a fresh address space with the arguments on the user stack and, on success, never returns. `exit(status)` stores the status and turns the process into a zombie (`ZOMBIE` state), and `wait(&status)` in the parent copies that status, frees the child and returns its PID. Unlike Linux, xv6 has no `PATH` search and no `WEXITSTATUS`: `wait` returns the exit value directly.

1.2 Pipes in xv6

`pipe(p)` creates a kernel object with a 512-byte circular buffer (`kernel/pipe.c`) and returns two descriptors: `p[0]` for reading and `p[1]` for writing. `piperead` sleeps while the buffer is empty and at least one writer exists; `pipewrite` sleeps while the buffer is full. These sleeps are what synchronise two processes without any explicit "wait for the other" code. A pipe carries bytes in one direction only.

2 RUN, INSIDE XV6

2.1 The Program

```c
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                    // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                        // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                // step 2: become echo
    // step 1.3:  char *args[] = { "ls", 0 };  exec("ls", args);
    // step 1.4:  exec("nosuchprog", args);
    printf("exec failed\n");           // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                       // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Listing 2: user/run.c

The two commented lines are the variants of steps 1.3 and 1.4; for each experiment I replaced the `args`/`exec` lines with them, left xv6 with `Ctrl-A X` and ran `make qemu` again.

2.2 Running It

```
$ make qemu
...
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

```
$
```

Listing 3: Step 1.2: the child becomes echo

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34232
...
run            2 25 34560
pingpong       2 26 34384
primes         2 27 34720
console        3 28 0
parent: child 4 ended with status 0
$
```

Listing 4: Step 1.3: the child becomes ls

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

Listing 5: Step 1.4: exec of a program that does not exist

Observations

1. Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Laboratory Work No. 7. What is the same?

Table 1: The three steps of the shell on both systems

Step	**xv6**	**Linux (Lab 7)**	**Notes**
1. copy	`fork()`	`fork()`	identical: 0 in the child, child PID in the parent
2. become	`exec("echo", args)`	`execlp("ls", "ls", "-l", NULL)`	xv6 takes an `argv` array and no `PATH` search
3. wait	`wait(&status)`	`waitpid(pid, &status, 0)`	xv6 returns the exit value directly, no `WEXITSTATUS`

The structure is the same: `fork` creates a copy and returns twice, the child calls `exec` and is replaced by another program (so the code after it runs only on failure), and the parent blocks in `wait` until the child exits and then receives its status. The program even has the same

shape and the same comments. Only the details of the library differ: Linux offers more convenient variants (`execlp` with `PATH` search, `waitpid` for a specific child), while xv6 has the bare system calls. `exec("echo", ...)` works in xv6 only because the shell's current directory is `/`, where all programs live.

2. Who prints "hello from the child": the code of `run`, or `echo`?

`echo` prints it. There is no such string as an output of `printf` in `run.c`: the words `hello`, `from`, `the`, `child` are only passed as `argv[1..4]` to `exec`. After `exec` the child process (PID 4) is no longer running the code of `run`; it runs `user/echo.c`, which writes its arguments separated by spaces and calls `exit(0)`. The same PID, a different program – that is why the parent then reports "child 4 ended with status 0".

3. Why is the status 1 here and 127 on Linux?

Because the status is simply the number that the child passes to `exit` after the failed `exec`, and the two programs pass different numbers: `user/run.c` calls `exit(1)`, while the Linux `run.c` calls `exit(127)`. The kernel does not invent the value; it only transports it from `exit` in the child to `wait` in the parent. `127` on Linux is the convention used by shells for "command not found"; xv6 does not follow that convention, and its own shell `sh.c` also just prints "exec failed".

3 PINGPONG

3.1 Implementation

The parent creates two pipes before `fork`, so both processes inherit all four ends. Each process first closes the two ends it does not use. The parent writes one byte into `p2c` (“ping”), the child reads it, prints its message, writes the byte back into `c2p` (“pong”) and exits; the parent reads the reply, prints its message and waits for the child.

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    if(read(p2c[0], &b, 1) != 1){       // blocks until the parent writes
      fprintf(2, "pingpong: child read failed\n");
      exit(1);
    }
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // send the same byte back
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);                 // ping
  if(read(c2p[0], &b, 1) != 1){         // blocks until the child replies
    fprintf(2, "pingpong: parent read failed\n");
    exit(1);
  }
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}
```

Listing 6: user/pingpong.c

```
$ run
...
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
```

```
$
```

Listing 7: Running pingpong (after one run of run, so the PIDs are 5 and 6)

The child (the higher PID) always prints "received ping" first, as required.

Observations

1. Why do you need TWO pipes, and not one for both directions?

A pipe is a single one-way buffer: whatever is written into `p[1]` can be read by *any* process holding `p[0]`, including the writer itself. With one pipe, after the parent writes the ping it would immediately call `read` on the same pipe and could take its own byte back before the child gets a chance to read it – then the child would wait forever and the parent would print "pong" without any reply. There is also no way to tell whose byte is in the buffer. Two pipes give each direction its own channel: the parent writes only to `p2c` and reads only from `c2p`, the child the other way round, so every byte can reach only the intended receiver. This is exactly why real systems use two pipes (or a socket) for two-way communication.

2. Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

The parent's `read(c2p[0], &b, 1)`. When the parent reaches it, the pipe `c2p` is still empty, so `piperead` in the kernel puts the parent to `sleep`; it is woken up only when the child's `write(c2p[1], &b, 1)` puts the byte into the buffer. In the same way the child's `read(p2c[0], &b, 1)` blocks until the parent has written the ping, which guarantees that "ping" is printed before "pong". The final `wait(0)` only collects the child's exit status; the synchronisation itself is done by the blocking `read`.

4 OPTIONAL: PRIMES

`primes.c` implements the sieve as a chain of processes. The main process writes the numbers 2..35 into the first pipe. Every stage reads its first number p – which must be prime, because all smaller primes already removed their multiples – prints it, creates a new pipe and a child for the next stage, and forwards only the numbers not divisible by p. When a stage gets end-of-file, it closes its right pipe (so the next stage also gets end-of-file), waits for its child and exits. Every pipe end that is not used is closed immediately, otherwise the chain would run out of the 16 descriptors per process and no stage would ever see end-of-file.

```c
#include "kernel/types.h"
#include "user/user.h"

// prime sieve with a chain of processes: every stage prints its first
// number p and forwards to the next stage only the numbers not divisible by p

void stage(int left) __attribute__((noreturn));

void
stage(int left)
{
  int p, n;

  if(read(left, &p, sizeof(p)) != sizeof(p)){
    close(left);                        // nothing came: end of the chain
    exit(0);
  }
  printf("prime %d\n", p);

  int right[2];
  pipe(right);
  if(fork() == 0){
    close(right[1]);                    // the next stage only reads
    close(left);
    stage(right[0]);
  }
  close(right[0]);                      // this stage only writes to the right
  while(read(left, &n, sizeof(n)) == sizeof(n)){
    if(n % p != 0)
      write(right[1], &n, sizeof(n));
  }
  close(left);
  close(right[1]);                      // EOF for the next stage
  wait(0);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int p[2];

  pipe(p);
```

```
44    if(fork() == 0){
45      close(p[1]);
46      stage(p[0]);
47    }
48    close(p[0]);
49    for(int i = 2; i <= 35; i++)
50      write(p[1], &i, sizeof(i));
51    close(p[1]);
52    wait(0);
53    exit(0);
54  }
```

Listing 8: user/primes.c

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$
```

Listing 9: Running primes

CONCLUSIONS

In this laboratory work I ran the program from Laboratory Work No. 7 inside xv6 and confirmed that the process model is the same in both systems: `fork` creates a copy, `exec` replaces the program and does not return on success, and `wait` lets the parent collect the child's exit status. The differences are only in the user library – no `PATH` search, no `WEXITSTATUS`, no `execlp` – and the exit status after a failed `exec` is whatever the programmer passes to `exit`: 1 in xv6, 127 on Linux. Seeing "hello from the child" printed by `echo` under the PID of the child made it clear that `exec` keeps the process but changes the program.

With `pingpong` I made two processes communicate in both directions. I learned why one pipe per direction is necessary, why every process must close the ends it does not use, and that the blocking `read` on an empty pipe is what synchronises the two processes without any explicit waiting code. The optional `primes` program extended the same idea to a whole chain of processes, where careful closing of pipe ends is essential both for termination and for staying within the limit of open files.